

Project 1: Warmup to C and Unix Programming – documentation

Student: Lauri Metso

Repository: <https://github.com/laurimetso/systeemiohjelmointi>

Summary

This project implements a simple program called **reverse**, written in C. The program reads files from an input source and prints them in reverse order to an output destination. It supports three invocation modes: reading from and writing to standard streams, reading from a file and writing to stdout or reading from one file and writing to another.

- `./reverse`: Reads from stdin and writes to stdout
- `./reverse input.txt`: Reads from file, writes to stdout
- `./reverse input.txt output.txt`: Reads from file and writes to file

How to compile

```
gcc reverse.c -o reverse
```

Program description

reverse

Reads lines from an input source into a dynamically growing array and prints them in reverse order to the output destination.

Usage:

```
./reverse
```

```
./reverse input.txt
```

```
./reverse input.txt output.txt
```

Implementation:

Command-line argument count is checked first. If more than two file arguments are provided, the program prints the usage message and exits with status 1. Input and output file/stream pointers are then configured based on argc.

Lines are read with `getLine()`, which allocates its buffer dynamically and therefore handles lines without any fixed size assumptions. Each line is duplicated with `strdup()` and stored in a pointer array. If the array's capacity is filled, it is doubled with `realloc()`, so long files are supported as well. Both operations check the return value and exit with status 1 printing "malloc failed" if allocation fails.

After all lines are read, the array is iterated in reverse and each line is printed with `fprintf()`. All allocated memory is freed and open file handles are closed before program returns 0.

Scored features

Feature	Points
Input and output are the same	Required, 1
File cannot be opened	Required, 0
Documentation	1
Additional assumptions	1
All specified errors addressed	2

Screenshots

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10 // määritetään rivien määrä, tää muuttuu vielä

int main(int argc, char *argv[]) {
    FILE *input_file = NULL; // tiedosto josta luetaan
    FILE *output_file = NULL; // tiedosto johon kirjoitetaan
    char *line = NULL; // pointeri stringeille
    size_t len = 0; // getline tarvitsee tämän
    char **lines = NULL; // null pointeri array mihin lisätään rivien pointerit
    int line_count = 0;
    int capacity = INITIAL_SIZE;

    if (argc > 3) {
        fprintf(stderr, "usage: reverse <input> <output>\n");
        exit(1);
    }

    // jos ei anneta filejä (argumentteja) niin luetaan stdin ja kirjoitetaan stdout
    if (argc == 1) {
        input_file = stdin;
        output_file = stdout;
    }

    // jos annetaan input file niin luetaan se ja kirjoitetaan stdout
    else if (argc == 2) {
        input_file = fopen(argv[1], "r");
        if (!input_file) {
            fprintf(stderr, "error: cannot open file '%s'\n", argv[1]);
            exit(1);
        }
        output_file = stdout;
    }

    // jos annetaan input ja output file niin luetaan input file ja kirjoitetaan output fileen.
    else if (argc == 3) {
        if (strcmp(argv[1], argv[2]) == 0) {
            fprintf(stderr, "Input and output file must differ\n");
            exit(1);
        }
        input_file = fopen(argv[1], "r");
        if (!input_file) {
            fprintf(stderr, "error: cannot open file '%s'\n", argv[1]);
            exit(1);
        }
    }
}
```

```
PS C:\Users\latem\Desktop\anytime\systemiohjelmointi\projekti1> gcc reverse.c -o reverse
PS C:\Users\latem\Desktop\anytime\systemiohjelmointi\projekti1> ./reverse
1
2
3
^Z
3
2
1
PS C:\Users\latem\Desktop\anytime\systemiohjelmointi\projekti1>
>X 0 0 0
```

<- ^Z is for

quitting the program.

```
PS C:\Users\latem\Desktop\anytime\systemiohjelmointi\projekti1> ./reverse input.txt output.txt
PS C:\Users\latem\Desktop\anytime\systemiohjelmointi\projekti1>
```